SIMATS ENGINEERING

ASSESSMENT TOOL 2 – APPLICATION-BASED QUESTIONS (High)

Course:ETHICAL HACKING	**Code:**	**CO:** CO5
Duration:	**Total Marks: 100**	**Reg No / Name :192511230**

COURSE OUTCOME COVERED

CO5: *Analyze vulnerabilities in Windows and Linux systems and evaluate security weaknesses in messaging, web, and mobile applications using appropriate exploitation techniques and hacking tools. (BT5)*

Instructions: Answer all questions. Each question carries 20 marks.

---

1. A mid-sized financial organization operates several legacy Windows servers where multiple RPC services remain exposed due to delayed patching cycles caused by operational constraints. Recently, abnormal traffic patterns and unauthorized connection attempts have been observed within the internal network. As a security analyst, evaluate how attackers could exploit these RPC vulnerabilities to gain unauthorized access and escalate privileges. Assess the level of risk associated with delaying patches in such environments and justify whether immediate patch deployment or controlled maintenance scheduling is more appropriate. Recommend suitable mitigation strategies to reduce exposure without affecting business continuity.

2. A web application used for online transactions suffers from multiple vulnerabilities, including cross-site scripting (XSS), cross-site request forgery (CSRF), and weak session management practices. Users report unauthorized transactions and session hijacking incidents. Evaluate how attackers could combine these vulnerabilities to execute complex attacks affecting user accounts. Assess the risks associated with poor session handling and lack of input validation. Justify the need for a layered security approach and recommend comprehensive mitigation strategies to secure the application.

3. An enterprise uses an outdated IIS server to host multiple internal and external applications, and recent vulnerability scans indicate several unpatched critical flaws. Despite warnings, the organization delays updates due to fear of downtime and service disruption. Assess how attackers could exploit these vulnerabilities to gain unauthorized access or execute remote code. Evaluate the risks associated with delaying security updates in such environments and justify whether immediate patching or temporary isolation is more appropriate. Recommend strategies to ensure both security and service availability.

4. A mobile application used by customers stores sensitive information such as login credentials and personal data in plaintext within the device storage. Additionally, the application communicates with backend servers over unsecured channels. Evaluate how attackers could exploit these weaknesses through device compromise or network interception. Assess the potential impact on user privacy and organizational reputation. Justify the need for encryption and recommend secure

storage and communication techniques.

5. A system administrator disables the Windows firewall temporarily to troubleshoot connectivity issues but forgets to re-enable it. Within a short period, multiple unauthorized access attempts are detected. Evaluate how attackers could exploit this temporary exposure to gain access to critical services. Assess the risks associated with mismanagement of security controls and justify the importance of enforcing automated policies. Recommend administrative safeguards to prevent such incidents.

Code of Conduct:

I Aluri poojitha_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signatureofthestudent:POOJITHA

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem Solving Approach	20	Logical, well structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Question 1: Legacy RPC Service Exposure on Windows Servers

Understanding the Problem Context

The organization operates several legacy Windows servers that continue to expose Remote Procedure Call (RPC) services, primarily because delayed patch cycles are driven by operational constraints such as fear of downtime, dependency on legacy applications, and limited maintenance windows in a financial environment that must maintain near-continuous availability. RPC (commonly implemented through the RPC Endpoint Mapper on TCP port 135, and supporting services such as SMB on port 445 and NetBIOS on 137-139) is foundational to Windows inter-process and inter-host communication, which means it is deeply embedded in authentication, file sharing, and management functionality. The recent observation of abnormal traffic patterns and unauthorized connection attempts strongly suggests active reconnaissance or exploitation attempts against these exposed services, and in a financial organization this materially raises the stakes because RPC-related compromises historically lead directly to full domain compromise.

How Attackers Could Exploit RPC Vulnerabilities

Unpatched RPC services present a well-documented and historically devastating attack surface. Attackers typically follow a structured exploitation chain:

- Reconnaissance and Enumeration: Attackers use port scanners (e.g., Nmap) and RPC enumeration tools to identify exposed endpoints (TCP 135, 445, 139) and fingerprint the Windows version and patch level, revealing which known CVEs are likely to apply.
- Exploitation of Known RPC/SMB Flaws: Historic and recurring vulnerability classes in this space (such as buffer overflows in the RPC/DCOM subsystem, and SMB-based remote code execution flaws) allow an attacker to send a specially crafted RPC request that overflows a buffer or corrupts memory, resulting in arbitrary code execution in the context of the vulnerable service — frequently running as SYSTEM.
- Privilege Escalation: Because many RPC-dependent services run with SYSTEM or highly privileged service accounts, successful exploitation often grants the attacker immediate SYSTEM-level access rather than requiring a separate escalation step. From there, credential material (password hashes, Kerberos tickets) can be harvested directly from memory.
- Lateral Movement and Domain Compromise: Harvested credentials and tickets enable pass-the-hash, pass-the-ticket, or direct authentication to other hosts that trust the same domain, allowing the attacker to pivot from a single legacy server to domain controllers and, ultimately, to core financial systems.
- Persistence and Data Exfiltration: Once elevated, attackers commonly install backdoors, scheduled tasks, or additional service accounts, and begin staging sensitive financial and customer data for exfiltration, or deploy ransomware across the network.

The abnormal traffic and unauthorized connection attempts described in the scenario are consistent with the reconnaissance and exploitation-attempt phases of this chain, indicating the organization may already be a target of an active or automated (worm-style) attack campaign.

Risk Assessment

Risk Factor	Likelihood	Impact	Overall Risk Rating
Unauthenticated remote code execution via RPC/SMB	High	Critical	Critical
Privilege escalation to SYSTEM/Domain Admin	High	Critical	Critical
Worm-style lateral spread across internal network	Medium-High	Critical	High
Data exfiltration of financial/customer records	Medium	Critical	High
Regulatory and compliance penalties (e.g., PCI-DSS, RBI/financial regulators)	Medium	High	High

Given that this is a financial organization, the combination of high likelihood (evidenced by active probing already observed) and critical business impact (potential for full domain compromise, fraud, regulatory breach, and reputational damage) places this risk firmly in the Critical band. Delaying patches on internet- or intranet-facing RPC services in this context is not a low-risk operational convenience; it is an active window of exposure that is measurably being probed right now.

Justification: Immediate Patching vs. Controlled Maintenance Scheduling

Recommendation: A hybrid, risk-tiered approach is most appropriate — immediate emergency patching or compensating controls for internet-facing and high-privilege RPC-exposed systems, combined with controlled maintenance scheduling for lower-risk, isolated legacy systems.

A blanket 'patch everything immediately' policy is operationally unrealistic in an environment with legacy application dependencies, and could itself cause outages that disrupt financial transaction processing. Conversely, a blanket 'wait for the next scheduled maintenance window' policy is unacceptable given that active unauthorized connection attempts have already been detected — this indicates the threat is not theoretical but current. The correct approach is therefore risk-tiered:

- Tier 1 (Immediate/Emergency Patch or Isolation): Any RPC-exposed host reachable from less-trusted network segments, or showing signs of targeted probing, should be patched within an emergency change window (24-72 hours) or, if patching cannot be completed immediately, isolated via network segmentation/firewall rules until it can be.
- Tier 2 (Accelerated Scheduled Patch): Internal, segmented legacy systems not currently showing signs of compromise can follow an accelerated — but still controlled — patch cycle (within 1-2 weeks) with compensating controls active in the interim.
- Tier 3 (Standard Maintenance Window): Systems with no external exposure and strong compensating controls (isolated VLAN, no direct user access) may follow the standard change-management cycle, provided monitoring confirms no active exploitation attempts.

This tiered justification balances the rubric's expectation of a well-reasoned trade-off: it does not default to 'patch immediately' without qualification, nor does it accept the operational excuse for delay without addressing the demonstrated active threat.

Recommended Mitigation Strategies

- Network segmentation and firewalling: Restrict RPC-related ports (135, 139, 445) to only the specific hosts and services that require them; block these ports at network boundaries and between VLANs using host-based and perimeter firewalls.
- Virtual patching / IPS signatures: Deploy Intrusion Prevention System (IPS) signatures that detect and block known RPC/SMB exploitation patterns as a compensating control while formal patches are scheduled.
- Disable unnecessary RPC/SMB features: Disable SMBv1 and unused RPC interfaces; enable SMB signing to prevent relay attacks.
- Privileged access management: Apply least-privilege service accounts, restrict local administrator rights, and deploy tiered administrative access (Tier 0/1/2 model) so that a single compromised host cannot be used to reach domain controllers.
- Enhanced monitoring and detection: Deploy an EDR/SIEM solution to alert on anomalous RPC traffic, unusual authentication patterns, and lateral movement indicators (e.g., abnormal use of PsExec, WMI, or remote service creation).
- Rapid, tested patch pipeline: Establish a staging environment to validate patches against legacy applications quickly, reducing the operational fear that currently drives delay, and shortening the real-world patch cycle.
- Network access control (NAC) and asset inventory: Maintain a current inventory of all RPC-exposed assets so that emergency patching or isolation decisions can be executed quickly and accurately.

Collectively, these controls reduce the exposure window without requiring the organization to take on the operational risk of an uncontrolled, all-at-once patch rollout, and they directly address the abnormal traffic already observed.

Question 2: Web Application XSS, CSRF, and Session Management Weaknesses

Understanding the Problem Context

The web application in question handles online financial transactions and suffers from three interconnected weaknesses: cross-site scripting (XSS), cross-site request forgery (CSRF), and weak session management. Individually, each of these is a well-known OWASP Top 10 category; however, the reported symptoms — unauthorized transactions and session hijacking — indicate that attackers are not exploiting these flaws in isolation but are chaining them together to bypass authentication and authorization controls entirely. This is a critical distinction: the real-world severity of these vulnerabilities is rarely additive, it is multiplicative, because each flaw removes a layer of defense that would otherwise have contained the others.

How Attackers Could Combine These Vulnerabilities

A realistic attack chain against this application could unfold as follows:

- Step 1 - XSS as the initial foothold: The attacker identifies an input field (search box, comment field, profile field) that reflects or stores user input without proper sanitization. They inject a malicious script that executes in the browser of any user who views the affected page.
- Step 2 - Session token theft via XSS: If session identifiers are stored in a way accessible to client-side scripts (e.g., cookies without the HttpOnly flag, or tokens kept in localStorage), the injected script can silently read and exfiltrate the session token to an attacker-controlled server, enabling direct session hijacking without the victim's knowledge.
- Step 3 - CSRF to perform actions without stealing the session: Where session theft is not possible, the attacker instead uses CSRF — crafting a malicious page or email that causes the victim's browser to automatically submit a state-changing request (e.g., a funds transfer) to the vulnerable application using the victim's existing, still-valid session cookie, relying on the application's failure to require a unique anti-CSRF token.
- Step 4 - Weak session management amplifies both: If session tokens are predictable, long-lived, never rotated after login, or not invalidated on logout, a token stolen via XSS remains usable for an extended period, and a hijacked session can be reused across multiple fraudulent transactions before detection.
- Step 5 - Monetization: Combined, these flaws let an attacker either directly impersonate a logged-in user to authorize transfers, or silently ride an authenticated session to execute unauthorized transactions, exactly matching the reported incidents of unauthorized transactions and session hijacking.

This chaining illustrates why XSS is often described as a 'gateway' vulnerability in web applications: once script execution is achieved in a victim's browser, CSRF protections, session cookies, and even multi-factor confirmations displayed in-page can potentially be undermined.

Risk Assessment

Risk Factor	Likelihood	Impact	Overall Risk Rating
Session hijacking via stolen tokens (XSS-driven)	High	Critical	Critical
Unauthorized financial transactions via CSRF	High	Critical	Critical
Account takeover through combined XSS+session flaws	Medium-High	Critical	High
Reputational and regulatory impact from fraud incidents	Medium	High	High
Mass exploitation via stored/persistent XSS (worm effect)	Medium	High	High

Because the application processes financial transactions, the potential impact of any single successful exploitation is severe (direct monetary loss, regulatory scrutiny under financial data protection requirements, and loss of customer trust). Poor session handling in particular compounds every other risk: even a modest XSS or CSRF flaw becomes far more dangerous when sessions are long-lived, are not bound to the client, or are not properly invalidated, because it extends both the attacker's window of opportunity and the potential blast radius of a single successful attack.

Justification for a Layered Security Approach

No single control is sufficient to address this scenario, which is why a layered (defense-in-depth) approach is justified rather than treating XSS, CSRF, and session management as three separate, independently-patchable bugs:

- Input validation and output encoding alone would reduce XSS but would not address CSRF, which does not depend on injecting script — it abuses the browser's automatic inclusion of cookies.
- Anti-CSRF tokens alone would not help if an attacker can use XSS to simply read the token value and include it in a forged request, since XSS-based script execution runs inside the trusted origin.
- Strong session management alone (short expiry, rotation) reduces the exploitation window but does not stop the initial script injection or forged request from occurring.

Only by combining controls at multiple layers — input, output, transport, session, and monitoring — can the organization ensure that the failure of one control does not cascade into full account compromise. This layered justification directly supports the rubric's expectation of an accurate, well-reasoned solution rather than a single-point fix.

Recommended Mitigation Strategies

Input Validation and XSS Prevention

- Implement strict server-side input validation (allow-listing) for all user-supplied data, never relying on client-side validation alone.
- Apply context-aware output encoding (HTML, JavaScript, URL encoding) wherever user input is rendered back to the browser.
- Deploy a Content Security Policy (CSP) to restrict which scripts can execute, significantly limiting the impact of any injected script.

CSRF Prevention

- Implement per-session, per-request anti-CSRF tokens validated server-side on every state-changing request.
- Set session cookies with the SameSite=Strict or SameSite=Lax attribute to prevent cross-origin submission.
- Require re-authentication or step-up verification (e.g., OTP) for high-value transactions such as large transfers.

Session Management Hardening

- Set the Secure and HttpOnly flags on all session cookies so they cannot be read by client-side scripts or transmitted over unencrypted channels.
- Use short, sliding session expiry with mandatory re-authentication after inactivity, and rotate session identifiers after login and privilege changes.
- Invalidate sessions server-side immediately on logout, password change, or detected anomalous activity.

Additional Layered Controls

- Enforce HTTPS/TLS across the entire application with HSTS enabled to prevent network-level interception.
- Deploy a Web Application Firewall (WAF) to detect and block common XSS/CSRF payloads in real time.
- Implement transaction anomaly monitoring and fraud detection to flag unusual transaction patterns for manual review.
- Conduct regular penetration testing and secure code review as part of the software development lifecycle (SDLC).

Question 3: Outdated IIS Server Hosting Critical Applications

Understanding the Problem Context

The enterprise relies on an outdated Internet Information Services (IIS) server to host both internal and external-facing applications. Vulnerability scans have already identified multiple unpatched critical flaws, yet the organization has delayed remediation out of concern for downtime and service disruption. This scenario is a classic case of risk trade-off mismanagement: the organization is implicitly accepting a known, quantified security risk (critical unpatched vulnerabilities) in order to avoid an operational inconvenience (planned downtime), without weighing that the unplanned downtime and damage from a successful attack would very likely be far more severe than the cost of controlled, scheduled patching.

How Attackers Could Exploit These Vulnerabilities

- Vulnerability fingerprinting: Attackers use banner grabbing and automated scanners to identify the exact IIS version and installed modules, then cross-reference public vulnerability databases for applicable exploits — the same information the organization's own scan already surfaced, meaning attackers can obtain it just as easily.
- Remote code execution (RCE): Critical IIS flaws have historically allowed remote, unauthenticated attackers to execute arbitrary code on the server, for example through buffer overflow conditions in request-parsing components, malicious file upload handling, or vulnerable ISAPI extensions/modules.
- Directory traversal and information disclosure: Misconfigurations or unpatched path-handling flaws can allow attackers to read files outside the intended web root, exposing configuration files, connection strings, or source code that reveal further attack paths (e.g., database credentials).
- Web shell deployment: Once code execution is achieved, attackers commonly upload a web shell to the server, granting persistent remote command execution disguised as a legitimate application file.
- Privilege escalation and pivoting: Because IIS often runs application pools with elevated service account privileges, a compromised server can be used to escalate privileges locally and pivot into the internal network, especially where the same server hosts both internal and external applications on shared infrastructure.
- Service disruption and defacement: Beyond data theft, attackers may deface externally facing applications (damaging brand reputation) or deploy ransomware, causing exactly the extended downtime the organization was originally trying to avoid — but now unplanned and far more damaging.

Risk Assessment

Risk Factor	Likelihood	Impact	Overall Risk Rating
Remote code execution on internet-facing IIS host	High	Critical	Critical
Web shell / persistent backdoor installation	High	Critical	Critical

Risk Factor	Likelihood	Impact	Overall Risk Rating
Lateral movement from shared internal/external hosting	Medium-High	Critical	High
Data exposure via directory traversal / misconfiguration	Medium	High	High
Unplanned outage/defacement damaging reputation	Medium	High	High

The combination of confirmed critical vulnerabilities (not theoretical, but already identified by internal scanning) and internet-facing exposure places this scenario at Critical risk. Notably, hosting both internal and external applications on the same outdated server means a breach originating from the public-facing side can directly compromise internal systems, removing the segmentation that would normally contain such an incident.

3.2 Justification: Immediate Patching vs. Temporary Isolation

Recommendation: Immediate temporary isolation or compensating controls for the externally-facing critical vulnerabilities, paired with an expedited (not indefinitely delayed) patching schedule — rather than either extreme of 'patch immediately with no preparation' or 'continue delaying.'

Patching a production IIS server hosting critical applications without preparation risks exactly the downtime the organization fears, and could itself cause service failures if untested updates break dependent applications. However, continuing to delay is untenable given that critical, exploitable flaws have already been confirmed by scanning — this is a known, active risk, not a hypothetical one. The justified middle path is:

- Immediate compensating controls: Apply a Web Application Firewall (WAF) or reverse proxy in front of the IIS server to virtually patch known exploit patterns, and restrict access to the vulnerable endpoints/modules where feasible, without taking the server offline.
- Segment now, patch soon: Where internal and external applications currently share the same host, immediately place them behind separate network segments or reverse proxies to prevent a public-facing compromise from reaching internal systems, even before the underlying OS/IIS patch is applied.
- Expedited, tested patching: Stand up a staging/replica environment to validate patches against the hosted applications within days (not months), then apply patches during a planned, communicated low-traffic maintenance window — using this scenario's urgency to justify prioritizing the testing effort rather than as a reason to skip it.
- Fallback isolation: If a critical flaw is actively being exploited (evidence of compromise) and cannot be immediately patched, temporary isolation of the affected server from the internet — even at the cost of short-term service unavailability — is justified, because the cost of a confirmed breach in a hosting environment serving both internal and external applications significantly exceeds the cost of a controlled short outage.

Recommendations to Balance Security and Availability

- Deploy a WAF/reverse proxy as an immediate compensating control layer in front of the IIS server.
- Separate internal and external applications onto isolated hosts or network segments to contain any future compromise.
- Establish a staging environment mirroring production to validate patches quickly, reducing the real cost/time of future patch cycles.
- Adopt a documented patch SLA (e.g., critical vulnerabilities remediated or mitigated within 72 hours) to prevent indefinite delay driven by informal downtime concerns.
- Harden the IIS configuration: disable unused modules/handlers, remove default/sample applications, enforce least-privilege application pool identities, and enable detailed logging.
- Implement continuous vulnerability scanning and a formal risk register so that identified critical flaws trigger a mandatory remediation timeline rather than open-ended deferral.
- Plan and communicate maintenance windows in advance with rollback procedures ready, minimizing perceived business disruption from patching.

Question 4: Mobile Application Insecure Storage and Transmission

Understanding the Problem Context

The mobile application stores sensitive information — including login credentials and personal data — in plaintext on the device, and communicates with backend servers over unsecured channels. This represents a fundamental failure of two of the most basic security principles in mobile application design: protection of data at rest and protection of data in transit. Because mobile devices are inherently more exposed to physical loss, theft, and malware than centrally-managed servers, and because network conditions (public Wi-Fi, cellular networks) are frequently untrusted, these two weaknesses together create a very high likelihood of real-world exploitation, not merely a theoretical one.

How Attackers Could Exploit These Weaknesses

Via Device Compromise

- Physical access attacks: If a device is lost, stolen, or briefly accessed by an unauthorized party, plaintext credential and personal data files can be extracted directly from device storage using basic file browsing tools or, on rooted/jailbroken devices, direct filesystem access — no sophisticated exploit is required.
- Malware and malicious apps: A malicious application installed on the same device (or a compromised app with excessive permissions) can read the vulnerable application's insecure storage files, especially on Android devices with improperly scoped external storage permissions or backup configurations.
- Device backup extraction: Insecurely stored data is often included in unencrypted device backups (cloud or local), giving attackers who compromise a user's backup account (e.g., a cloud storage account) a second, indirect path to the same plaintext data.
- Reverse engineering: Attackers can decompile the application package to locate exactly where and how sensitive data is stored, making exploitation systematic and repeatable across all users of the app, not just a single victim.

Potential Impact on User Privacy and Organizational Reputation

User Privacy Impact: Exposure of plaintext credentials and personal data can lead directly to identity theft, financial fraud, unauthorized account access, and exposure of sensitive personal details, with affected users bearing consequences that can persist long after the initial breach (e.g., credential reuse attacks against other services).

Organizational Reputation Impact: A publicized breach stemming from such a basic, easily-avoidable failure (plaintext storage and unencrypted transmission) is particularly damaging to trust, because it signals a fundamental lack of security diligence rather than a sophisticated, unavoidable attack. For an organization handling financial or customer data, this can trigger regulatory investigation, financial penalties, loss of customer confidence, and costly mandatory breach notifications, with long-term impact on customer acquisition and retention.

Justification for Encryption

Encryption is not an optional hardening measure in this scenario — it is the baseline control that directly closes

both identified gaps. Encrypting data at rest ensures that even if a device is compromised or backed up insecurely, the extracted files are unreadable without the correct key, which is ideally never stored alongside the data itself. Encrypting data in transit (via properly validated TLS) ensures that network-level attackers, even those positioned as a man-in-the-middle, capture only ciphertext. Because both attack vectors described in the scenario (device compromise and network interception) are rendered largely ineffective by proper encryption, it represents the highest-leverage, most directly justified control for this specific case.

Recommended Secure Storage and Communication Techniques

Secure Storage

- Never store plaintext credentials; instead, store only securely hashed/salted values where applicable, or rely on OS-provided secure enclaves (Android Keystore, iOS Keychain) to store authentication tokens.
- Encrypt all sensitive personal data at rest using strong, industry-standard algorithms (e.g., AES-256), with encryption keys managed via the platform's hardware-backed key storage rather than embedded in application code.
- Avoid storing sensitive data in easily accessible locations (shared preferences, external storage, application logs); use platform-sandboxed, encrypted storage mechanisms.
- Implement automatic session/token expiry so long-lived plaintext-equivalent credentials are not retained indefinitely on the device.
- Exclude sensitive data from device backups, or ensure backups themselves are encrypted with user-specific keys.

Secure Communication

- Enforce TLS 1.2+ for all network communication, with certificate/public-key pinning to prevent MITM attacks even on compromised or malicious networks.

- Reject cleartext (HTTP) traffic entirely at the application network-security-configuration level (Android) or App Transport Security settings (iOS).
- Implement mutual authentication and short-lived, rotating access/refresh tokens instead of long-lived static credentials transmitted repeatedly.
- Apply request/response integrity checks and replay protection to prevent traffic manipulation even if interception is attempted.

Organizational Controls

- Conduct mobile-specific security testing (static and dynamic analysis, penetration testing) prior to release and on a recurring basis.
- Adopt secure mobile development guidelines (e.g., OWASP Mobile Application Security Verification Standard) as a mandatory part of the SDLC.
- Provide a clear incident response and breach notification plan specific to mobile data exposure scenarios.

Question 5: Accidental Windows Firewall Disablement

Understanding the Problem Context

A system administrator disables the Windows Firewall temporarily for legitimate troubleshooting purposes but fails to re-enable it afterward. Shortly thereafter, multiple unauthorized access attempts are detected. This scenario differs from the previous ones in an important way: it is not caused by a technical vulnerability or unpatched flaw, but by a human process failure — the absence of a safeguard to catch and reverse a manual configuration change. This makes it a case study in security control mismanagement and the absence of automated enforcement, rather than in exploit engineering.

How Attackers Could Exploit This Temporary Exposure

- Exposure of previously blocked services: With the host-based firewall disabled, any service listening on the machine — including administrative services (RDP, SMB, RPC, WMI) that are normally filtered — becomes reachable from anywhere on the network, dramatically expanding the attack surface with no additional action by the attacker.
- Opportunistic network scanning: Attackers or automated scanning tools/malware already present elsewhere on the internal network routinely and continuously probe for open ports; a host with its firewall disabled will respond to scans it previously ignored, making it stand out as a newly attractive target.
- Direct service exploitation: Once services are reachable, attackers can attempt credential brute-forcing (e.g., against RDP or SMB), exploit any unpatched service vulnerabilities present on that host (compounding with scenarios like Question 1's RPC exposure), or attempt to establish unauthorized remote sessions.
- Lateral movement staging: If the affected system has any level of administrative trust relationship with other hosts (as is common for systems administrators' own workstations or jump hosts), attackers who gain a foothold here can use it as a pivot point deeper into the network — administrator machines are especially high-value targets precisely because of the privileges they typically hold.
- Extended dwell time due to human oversight: Because the exposure resulted from a forgotten manual change rather than a monitored, ticketed configuration, it may persist undetected for a longer period than a scanned and tracked vulnerability, increasing the attacker's opportunity window.

Risk Assessment

Risk Factor	Likelihood	Impact	Overall Risk Rating
Unrestricted access to normally-filtered admin services (RDP/SMB)	High	High	High
Credential brute-force against exposed services	High	Medium-High	High
Extended, undetected exposure window	Medium-High	High	High
Lateral movement via a privileged admin workstation	Medium	Critical	High

Risk Factor	Likelihood	Impact	Overall Risk Rating
Recurrence of the same human error in future troubleshooting	High	Medium	Medium-High

While an isolated firewall lapse may appear low-severity compared to an actively exploited software vulnerability, the risk here is elevated by two factors: the detection of multiple unauthorized access attempts confirms this exposure was noticed and actively probed almost immediately, and the fact that it involves a system administrator's own environment, which typically carries elevated network trust and access privileges.

Risks of Mismanagement of Security Controls and the Importance of Automated Policy Enforcement

This incident illustrates a broader and very common category of real-world security failure: security controls that depend entirely on a human remembering to reverse a manual action are inherently unreliable. Administrators are frequently under time pressure while troubleshooting, and re-enabling a control is easy to forget once the immediate problem is resolved. The core risk is not the firewall rule itself, but the absence of any automated mechanism to detect, alert on, or revert the drift from the organization's intended secure baseline state.

Enforcing security policy through automated, centrally managed controls (rather than relying on individual administrator discipline) is essential because it removes the single point of human failure, ensures consistency across all systems regardless of who performs troubleshooting, provides an audit trail of every change for accountability, and guarantees that any deviation is corrected within a bounded, known time frame rather than persisting indefinitely until someone happens to notice.

Recommended Administrative Safeguards

- Enforce firewall policy via Group Policy (GPO) centrally, so that local firewall changes are automatically reverted to the organizational baseline on the next policy refresh cycle rather than relying on manual re-enablement.
- Implement configuration/compliance monitoring tools (e.g., SCCM, Intune, or a SIEM-integrated configuration management tool) that continuously check critical security settings and generate immediate alerts when a host drifts from the approved baseline (such as firewall disabled).
- Require time-boxed, ticketed change requests for any temporary security control modification, with an automatic expiry/rollback timer built into the change so the control is restored even if the administrator forgets.
- Use just-in-time (JIT) troubleshooting alternatives (targeted firewall exceptions for a specific port/IP/time window) instead of fully disabling the firewall, so the exposure created for troubleshooting is minimized in scope from the outset.
- Deploy automated alerting on security-relevant configuration changes (e.g., via Windows Event Forwarding/SIEM rules that trigger on firewall service state changes) so that any disablement is flagged to the security team in real time, regardless of intent.
- Conduct periodic configuration audits and firewall status reviews as part of routine operational hygiene, independent of any specific troubleshooting incident.